

Hochschule Harz

Fachbereich Automatisierung und Informatik

Spezialisierung: Virtuelle Welten - Anwendungspraktikum

Prof. Dr.-Ing. Simon Adler

HIKE

Harzer Interaktives Karten Erlebnis

Abschlussdokumentation

vorgelegt von:

Matthias Mersdorf
m29577
u37013@hs-harz.de

Fabian Siemens
m29620
u37052@hs-harz.de

Studiengang:
Fachsemester:

Informatik
6

Erstprüfer:
Zweitprüfer:

Prof. Dr.-Ing. Simon Adler
Sebastian Kliner

Ort, Datum:

Wernigerode, 19.07.2024

I. Inhaltsverzeichnis

I.	Inhaltsverzeichnis	2
II.	Abbildungs- /Tabellenverzeichnis	4
1	Über das Projekt	5
1.1	Projektidee	5
1.2	Team	5
1.3	Projektziele	5
1.3.1	Verpflichtende Ziele	5
1.3.2	Optionale Ziele.....	6
1.4	Ursprünglicher Zeitplan	6
1.5	Tatsächlicher Zeitplan.....	7
2	Umsetzung.....	8
2.1	Verwendete Technologien, Bibliotheken und Erweiterungen.....	8
2.2	Projektarchitektur	8
2.3	Harzer 3D Modell.....	9
2.3.1	Datengrundlage	9
2.3.2	Darstellung.....	9
2.3.3	Optimierung.....	11
2.4	Stempelstellen	12
2.4.1	Modell.....	13
2.4.2	Datengrundlage	13
2.4.3	Platzierung	14
2.4.4	Interaktion	15
2.5	Wanderrouten	15
2.5.1	Datengrundlage	15
2.5.2	Darstellung und Platzierung	16

2.5.3	Interaktion	16
2.6	Umgebung	16
3	Ergebnis	17
3.1	Zielumsetzung.....	17
3.2	Inbetriebnahme	17
3.3	Bedienung.....	18
3.4	Konfigurationsmöglichkeiten	18
4	Fazit.....	21
4.1	Herausforderungen	21
4.2	Eigenreflektion.....	22
4.2.1	Fabian	22
4.2.2	Matthias.....	23
4.3	Ausblick.....	24
III.	Quellen- / Ressourcenverzeichnis	25
IV.	Anhang.....	25

II. Abbildungs- /Tabellenverzeichnis

Abbildung 1: Ordnerstruktur des Unity-Projekts	8
Abbildung 2: Verwendete Segmente des DGMs	9
Abbildung 3: Harz-DGM Darstellung in Unity.....	10
Abbildung 4: Finale DGM-Darstellung mit Texturen in Unity	11
Abbildung 5: Speichernutzung von Unity beim Generieren des Modells mit der alten Methode. Der hellblaue Balken ist ungenutzter reservierter Speicher.	12
Abbildung 6: Ansicht des in Blender 4.0 erstellten Modells eines Stempelstellen-Kastens	13
Abbildung 7: Finale Darstellung der Stempelstellen auf dem Modell in Unity.....	14
Abbildung 8: Interaktion mit Stempelstellen	15
Abbildung 9: Finale Darstellung der Wanderrouten auf dem Modell in Unity.....	16
Abbildung 10: Ausschnitt aus dem Einstellungsfenster in Unity. Zu sehen ist der HikeSettingsProvider mit allen Konfigurationsmöglichkeiten.	18
Abbildung 11: Unterschied zwischen Big Endian (links) und Little Endian (rechts)	21
Tabelle 1: Umsetzungsstatus der Projektziele (✓ = vollständig erfüllt, * = teilweise erfüllt, ✕= nicht erfüllt)	17
Tabelle 2: Einstellungen in Unity im HikeSettingsProvider	19

1 Über das Projekt

1.1 Projektidee

Die Idee des Projekts „Harzer interaktives Karten-Erlebnis“ (kurz „HIKE“) war, dem Nutzer einen Überblick der 222 Stempelstellen der „Harzer Wandernadel“ zu ermöglichen. Diese sollten zusammen mit einer Auswahl von Wanderwegen auf einem realitätsnahen Modell des Harz-Mittelgebirges mit Höhenrelief dargestellt werden. Weitere Idee des Projekts war es, eine Nutzerinteraktion mit den dargestellten Wanderwegen und Stempelstellen zu ermöglichen, um deren Informationen darzustellen und filtern zu können. Für eine verbesserte Anschaulichkeit entstand die Idee, Google Street View für Einblicke am gewählten Standort zu nutzen.

1.2 Team

Das Team für das HIKE-Projekt bestand aus Fabian Siemens und Matthias Mersdorf, beide Studenten im sechsten Semester des Studiengangs Informatik 2021 an der Hochschule Harz.

1.3 Projektziele

Die Ziele des Projekts wurden in verpflichtende und optionale Ziele aufgeteilt, wobei erstere bis zum Ende des Projekts erreicht werden mussten. Für die optionalen Ziele galt der gleiche Zeitrahmen, allerdings wurden sie als zusätzliche Ziele betrachtet, die erreicht werden könnten.

1.3.1 Verpflichtende Ziele

Die fünf verpflichtenden Ziele des Projekts waren:

- Erstellung eines dreidimensionalen Modells des Harzes
- Anzeigen der Wanderwegen auf dem Harz-Modell
- Anzeigen der Stempelstellen auf dem Harz-Modell
- Interaktion mit Wanderwegen für Darstellung der betreffenden Informationen
- Interaktion mit Stempelstellen für Darstellung der betreffenden Informationen

1.3.2 Optionale Ziele

Die zwei optionalen Ziele des Projekts waren:

- Einbinden von Google Street View
- Färbung der im Modell dargestellten Wanderrouen je nach Schwierigkeitsgrad

1.4 Ursprünglicher Zeitplan

Für die Bearbeitung des Projekts war ein Zeitrahmen von 11 Wochen vorgesehen. Der Projektstart erfolgte mit der Kick-Off-Präsentation am 4. April 2024 und die Abschlusspräsentation, und somit das reguläre Ende des Bearbeitungszeitraums, war am 21.06.2024. In jeder Arbeitswoche stand dem Team ein kombiniertes Zeitbudget von 8 Stunden (also 4 Stunden pro Person) zur Verfügung. Durch Verrechnung mit dem Effektivitätsfaktor 0.5, da beide Teilnehmer unerfahren im Umgang mit den verwendeten Technologien waren, belief sich die Anzahl der effektiven Stunden (Mannstunden) pro Woche auf 4. Das Projekt wurde daraufhin in Aufgabenpakete unterteilt, welchen jeweils eine Dauer in effektiven Mannstunden, also die Zeit, die ein erfahrener Entwickler benötigen würde, zugeteilt wurde. Der vollständige Zeitplan ist im Anhang unter „Zeitplan_Original_Anwendungspraktikum_XR.xmls“ zu finden. Im Folgenden wird der Zeitplan kurz beschrieben. Der Zeitplan sah als erstes Aufgabenbündel die Akquisition der Daten, wie bspw. Höhendaten, Stempelstellen vor. Die Höhendaten sollten daraufhin konvertiert und in Unity dargestellt werden, gleiches sollte daraufhin mit den Stempelstellen erfolgen. Danach war die Implementierung der VR-Steuerung und graphischer Nutzeroberflächen angedacht, gefolgt von der Implementierung von Texturen und Platzierung von Assets, wie bspw. Bäumen. Nach dem auch die Interaktion mit den Stempelstellen erledigt ist, sollten die Wanderrouen-Daten akquiriert, importiert und dargestellt werden. Als letzten verpflichtenden Schritt folgte in dem ursprünglichen Zeitplan die Wanderrouen-Interaktion. Daraufhin waren nur noch optionale Ziele, wie die Implementierung von Google Street View geplant. Parallel sollte der Projektfortschritt dokumentiert werden. Pufferzeiten, bzw. Zeiträume für Bugfixes waren nicht angedacht, was sich rückblickend als Fehler erwies. Auf diesen Umstand wird in Kapitel 4 genauer eingegangen.

1.5 Tatsächlicher Zeitplan

Der Zeitplan wurde im Laufe des Projekts unter Berücksichtigung des aktuellen Fortschritts und bestehender Komplikationen angepasst. Dadurch wurde der originale Zeitplan substantiell geändert. Auf diese Änderungen soll in diesem Kapitel eingegangen werden, ohne dabei auf die Hintergründe einzugehen. Die Aufarbeitung der Komplikationen und Umstände erfolgt in den Kapiteln 4.1 bzw. 4.2. Die erste Adaption war das Tauschen der Erstellung von Assets (was sich auf das Stempelstellenmodell beschränkte) und der Erstellung der Modell-Texturen. Ersteres wurde vorgezogen. Die automatisierte Erstellung der Texturen erwies sich als komplexer als gedacht und dauerte zwölf effektive Arbeitsstunden, anstelle der geplanten vier. Gleichzeitig wurde die Implementierung der VR-Steuerung und Stempelstelleninteraktion um mehr als ein Monat verschoben. Das optionale Ziel Google Street View und ein geplantes Auswahlmenü für Wanderrouen wurden verworfen. Der vollständige überarbeitete Zeitplan ist im Anhang unter „Zeitplan_Final_Anwendungspraktikum_XR.xlms“ zu finden.

2 Umsetzung

2.1 Verwendete Technologien, Bibliotheken und Erweiterungen

Zur Umsetzung des Projekts wurde eine Palette von verschiedenen Programmen, Technologien und Bibliotheken verwendet. Die direkte Umsetzung erfolgte in Unity 2022.3.22f1 mit dem Universal Render Pipeline 3D Template. Das Projekt wurde mit GIT verwaltet und auf GitHub hochgeladen. Alle dargestellten 3D Modelle wurden entweder in Unity erstellt bzw. automatisch generiert, oder in Blender 4.0 erstellt. Zur Visualisierung in VR wurden die Unity Pakete „XR Interaction Toolkit“, sowie „XR Plugin Management“ hinzugefügt. Als VR-Brille wurde eine HP Reverb verwendet. Die Unity Pakete „Splines“ und „Text Mesh Pro“ wurden ebenfalls genutzt. Wie in den folgenden Kapiteln beschrieben, wurden einige Python-Skripte zur Vorverarbeitung von Daten verwendet, wofür Python 3 mit einigen spezifischen Bibliotheken, wie „PIL“, „bs4“ und „pyproj“ genutzt wurde.

2.2 Projektarchitektur

Das Projekt besteht aus einer einzigen Szene, welche u.a. die GameObjects „TerrainManager“, „StempelstellenManager“, „WanderroutenManager“, „Table“, „TextFields“ und „XR Interaction Manager“ enthält. Der TerrainManager enthält als Kindobjekte die Modell-Segmente vom Harz-Modell, auf welches in Kapitel 2.3 eingegangen wird. Der StempelstellenManager funktioniert analog zum TerrainManager für die Stempelstellen, genauso wie der WanderroutenManager für Wanderrouten. Diese drei Manager haben enthalten im Unity-Editor die Knöpfe „Regenerieren“, „Clear“ und „Regenerate on Game Start“. Auf die Funktion dieser Knöpfe wird in Kapitel 3.2 eingegangen. Die Dateistruktur des Projekts ist der nebenstehenden Abbildung zu entnehmen.

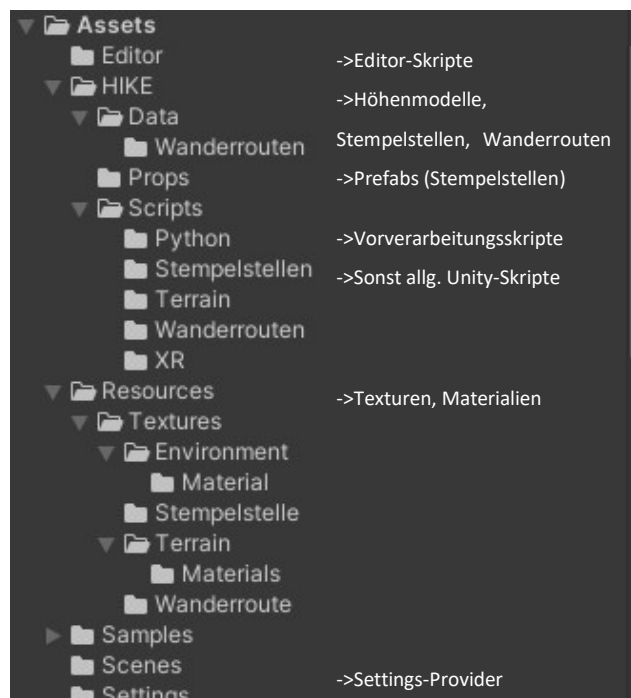


Abbildung 1: Ordnerstruktur des Unity-Projekts

2.3 Harzer 3D Modell

Als Kernkomponente von HIKE, stellte die Erstellung der dreidimensionalen Karte des Harzes den ersten Schritt der Projektumsetzung dar. Um ein möglichst realistisches Ergebnis zu erzielen, wurde eine solide, detaillierte Datengrundlage benötigt:

2.3.1 Datengrundlage

Als Quelle kamen viele verschiedene Institutionen, welche über Höhendaten des Harzes verfügen, in Frage. Beispielsweise die ESA, ArcGIS, das Copernicus Programm, sowie die Landesvermessungsbehörden der Länder Sachsen-Anhalt, Niedersachsen und Thüringen. Letztendlich verfügte aber nur das Landesamt für Vermessung und Geoinformation (LVermGeo) über ausreichend detaillierte kostenlose Höhendaten. Ausgewählt wurde das Digitale Geländemodell (DGM) von Sachsen-Anhalt, in der Gitterweite 5 Meter. Das Geländemodell besteht aus

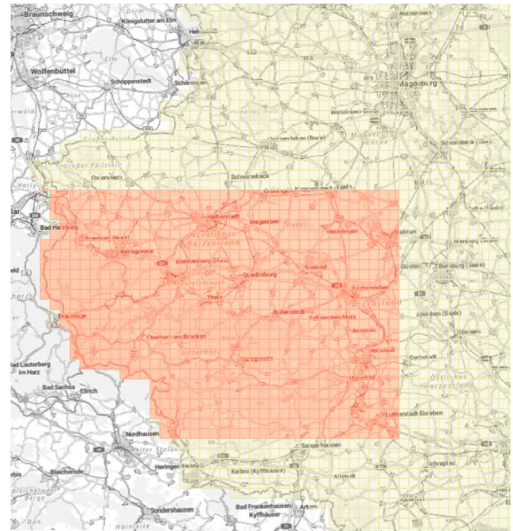


Abbildung 2: Verwendete Segmente des DGMs

5466 Segmenten, welche jeweils maximal 400 x 400 Samplepunkte enthalten. Die Segmente liegen als XYZ-Dateien vor, also ASCII kodierte Textdateien, wo die Koordinaten (X, Y), gefolgt von der Höhe über Normalnull des Samplepunkts als komma-separierte Werte dargestellt werden. Die Koordinaten nehmen Bezug auf das Europäische Terrestrische Referenzsystem 1989 (ETRS89), welches in der UTM-Zone 32U liegt. Leider enthält dieser Datensatz nur Höhendaten für Sachsen-Anhalt. Niedersachsen und Thüringen stellten im Projektzeitraum keine kostenlosen Geländemodelle zur Verfügung, weshalb der Harz in diesen Ländern nicht dargestellt wurde. Um nur die für das Projekt relevante Harzer Region abzubilden, wurde das DGM auf die in Abbildung 2 dargestellte Fläche von 36x25 (theoretischen) Segmenten beschränkt. Die tatsächliche Anzahl der Segmente liegt allerdings darunter, da, wie zu sehen ist, einige Segmente zu Niedersachsen gehören.

2.3.2 Darstellung

Zur Darstellung des DGMs in Unity gab es verschiedene Möglichkeiten. Die erste explorierte Variante war die Darstellung über ein Mesh, was im Vorfeld über eine andere Software, wie

bspw. MeshLab generiert und in Unity importiert wird. Diese Variante funktionierte gut für kleine Segmente, allerdings nicht für das vollständige Harz-DGM. Die zweite probierte und auch finale Variante der Darstellung ist die Verwendung von Unitys Terrain-Objekten. Terrains sind spezielle Ebenen in Unity, welchen eine Heightmap zugeteilt wird. Diese Heightmap enthält für alle Samplepunkte des Terrains einen Höhenwert zwischen 0 (minimale Höhe) und 1 (maximale Höhe). Dem Objekt muss dann eine Auflösung $r = 2n + 1$ für $n \in \mathbb{N} \cap [5, 12]$, also 33 bis 4097, sowie beliebige Maße, für Länge, Breite und Höhe zugewiesen werden. Terrains haben den Vorteil, dass sie ein eingebautes Level of Detail haben, Unity also die Terrainauflösung bei hoher Distanz zur Kamera verringert. Da das Modell vom Harz $36 \times 400 = 14400$ Samplepunkte auf seiner längsten Seite hat und ein Terrain maximal 4097 Samplepunkte breit sein kann, werden mit jeder Terrainauflösung mehrere Terrain-Objekte benötigt, um das gesamte Modell vom Harz abzubilden. Um die Abbildung möglichst maßstabsgetreu und ohne Verzerrungen darstellen zu können, müssen die Terrains also die Maße $t = r * w$ haben, wobei w die Gitterweite des DGMs ist. Die Terrainauflösung r entscheidet in diesem Fall also nur noch über die Anzahl der zur Darstellung genutzten Terrains. Hierfür muss allerdings das DGM als vollständige zweidimensionale Matrix von Höhenwerten vorliegen, im Gegensatz zu den bisherigen einzelnen Segmenten, um dann auf die Terrains aufgeteilt werden zu können. Das Fusionieren der DGM-Segmente zu einer einzelnen Datei wurde über das Python Skript „dgmConverter.py“ umgesetzt, welches im Ordner Assets/HIKE/Scripts/Python ist. Der Prozess war mit vielen Problemen übersät und

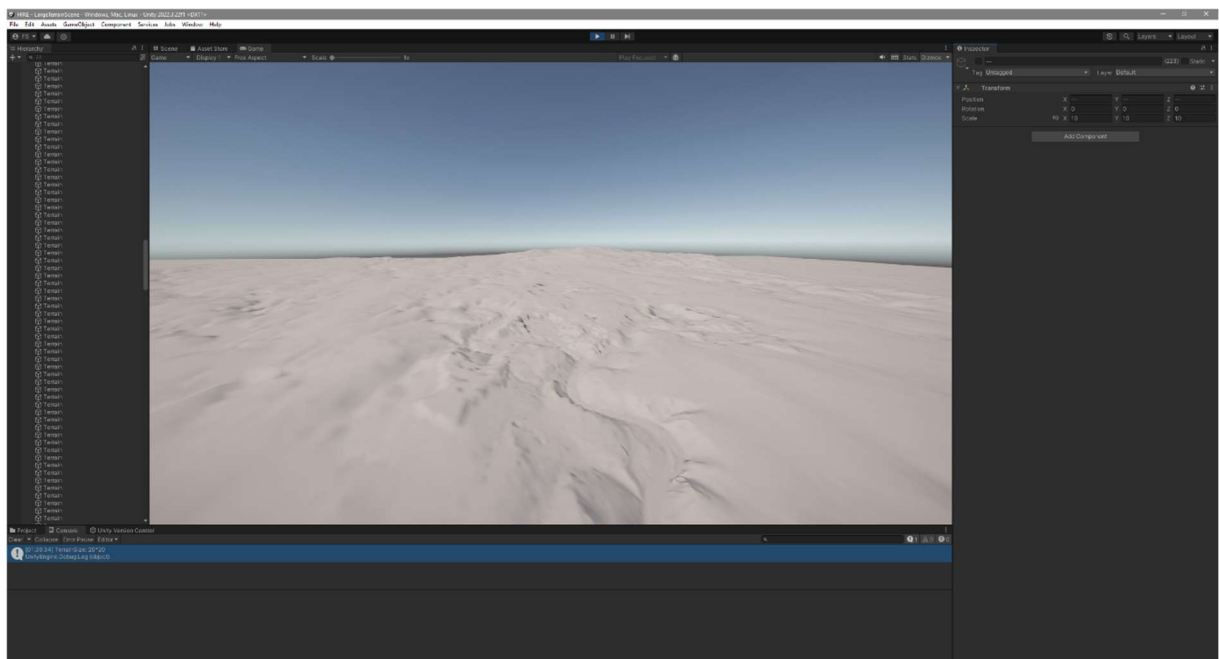


Abbildung 3: Harz-DGM Darstellung in Unity

es bedurfte vieler Optimierungen, welche in den Kapiteln 2.3.3 bzw. 4.1 beschrieben werden. Abbildung 3 zeigt einen Screenshot aus Unity mit fertigem Terrain. Um dem Modell eine Textur zu geben, werden Überflugbilder des LVerGeo genutzt, welche im tiff-Format vorliegen. Diese Bilder müssen auf eine bestimmte angepasst werden. Hierfür wurden die Überflugbilder in dem Python-Skript „textureMerger.py“ zu einem großen Bild zusammengeführt und dann in passend große Segmente aufgeteilt. Das fertige Modell, wie es auch in der Abgabeverision zu sehen ist, ist auf folgendem Bild abgebildet.



Abbildung 4: Finale DGM-Darstellung mit Texturen in Unity

2.3.3 Optimierung

Das ursprüngliche DGM von Sachsen-Anhalt hatte eine Größe von 3,7GB komprimiert und ~20GB dekomprimiert. Die für den Harz relevanten DGM-Segmente belaufen sich auf ~4GB. Ursprünglich wurden diese kompletten 4GB beim Generieren der Terrains in eine zweidimensionale Matrix eingelesen und dann auf Terrains aufgeteilt. Diese Methode führte allerdings dazu, dass Unity in großen Mengen Arbeitsspeicher reserviert, was in einer RAM-Auslastungen von über 90% (ca. 25GB für Unity) resultierte. Um diese Problematik zu reduzieren, wurde das DGM mithilfe des Python-Skripts „dgmConverter.py“ vorverarbeitet. Dabei werden alle Höhenwerte des DGMs in Werte zwischen 0 und 1 umgerechnet und in ein einzelnes zweidimensionales Array überführt, welches den Maßen des Harzer DGM-Ausschnitts entspricht. Eine vom Skript generierte Index-Datei enthält dabei Informationen wie minimale und maximale Höhe, sowie Länge und Breite des DGMs. Das Array mit den

Höhendaten wird daraufhin als eine Bytes-Datei exportiert. Diese Variante reduziert den benötigten Festplattenspeicher auf ca. 560MB und den Hauptspeicherbedarf auf ~1GB.

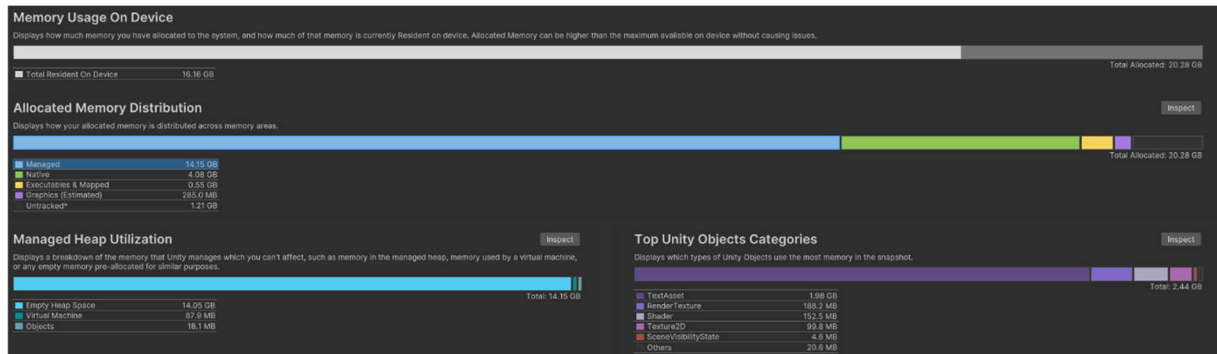


Abbildung 5: Speichernutzung von Unity beim Generieren des Modells mit der alten Methode. Der hellblaue Balken ist ungenutzter reservierter Speicher.

2.4 Stempelstellen

Auf dem Harz-Modell sollen die Stempelstellen der Harzer Wandernadel als Miniaturen dargestellt werden. Dafür müssen die dreidimensionalen Miniaturen der Stempelstellen-Kästen an der korrekten Stelle im Harz-Modell platziert werden. Korrekt bedeutet in diesem Fall, dass der Standort des Stempelstellen-Kastens im Harz-Modell der Koordinate der realen Stempelstelle entspricht. Die Positionierung soll also maßstabsgetreu sein und dem Verhältnis Modellmaßstab - Realität entsprechen.

2.4.1 Modell

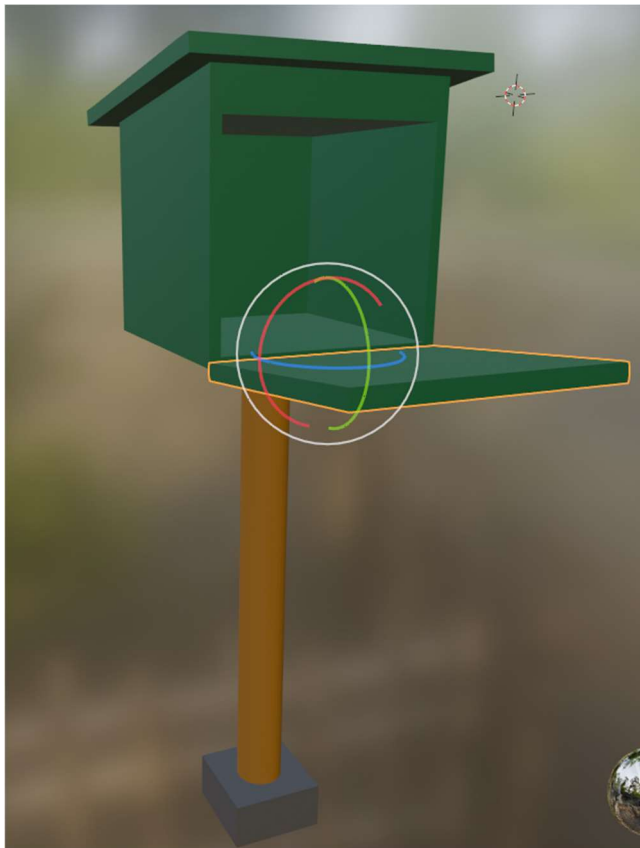


Abbildung 6: Ansicht des in Blender 4.0 erstellten Modells eines Stempelstellen-Kastens

Das Modell des Stempelstellen-Kastens wurde mithilfe von Blender 4.0 erstellt. Die Maße dieses Modells entsprechen denen des Kastens der Stempelstelle auf dem Agnesberg in Wernigerode. Dieser Kasten ist die Stempelstelle Nummer 31 der Harzer Wandernadel und wurde für eine maßstabsgetreue Modellierung vermessen. Da alle regulären Stempelstellen-Kästen (also ohne die Sonderstempel-Kästen) äußerlich baugleich und gleichfarbig sind, reichte die Erstellung eines Modells. Im Anschluss daran wurde das Modell der Stempelstelle mit einer Animation zum Öffnen der Vorderklappe versehen. Das schlussendlich verwendete Modell mit

geöffneter Vorderklappe ist in Abbildung 6 zu sehen.

2.4.2 Datengrundlage

Die Koordinaten der Stempelstellen-Kästen der Harzer Wandernadel wurden aus der Quelle [1] entnommen und mussten in ein verwertbares Format gebracht werden. Hierfür wurde die Internetseite mit der tabellarischen Auflistung der Stempelstellen als .htm-Datei gespeichert und die Code-Zeile mit den Stempelstellen als .txt-Datei herauskopiert. Danach wurde mithilfe eines C++-Programms aus einer Zeile, bestehend aus html-Tags und den Stempelstelleninformationen, eine .txt-Datei erzeugt. In dieser Datei befanden sich durch Kommata getrennt zeilenweise die 222 Stempelstellen mit ihren Eigenschaften. Die Erstellung dieser Datei erfolgte deshalb mittels C++, weil diese Aufgabe einer Übungsaufgabe aus einer C++-Lehrveranstaltung ähnelte.

2.4.3 Platzierung

Damit die Stempelstellenmodelle an den passenden Punkten des Modells platziert werden können, müssen die Koordinaten der Stempelstellen, welche im WGS84-Format vorlagen, in



Abbildung 7: Finale Darstellung der Stempelstellen auf dem Modell in Unity

ETRS89 Koordinaten umgerechnet werden. Dieser Schritt erfolgt in der Vorverarbeitung der Daten, wobei mit dem Python-Skript „processStempelstellen.py“ die kommaseparieren Werte in ein JSON-Konstrukt übertragen und die Koordinaten in ETRS89 umgewandelt wurden. In Unity wird die JSON-Datei eingelesen und zu C# Objekten deserialisiert. Das Prefab der Stempelstelle wird daraufhin für jede Stempelstelle instanziiert und an den passenden Unity-Koordinaten platziert. Die Unity-Koordinaten werden vom CoordinateService berechnet, welcher die ETRS89-Koordinaten an den Skalierungs- und Überhöhungsfaktor des Modells anpasst.

2.4.4 Interaktion

Die Interaktion mit den Stempelstellen beschränkt sich auf Grund von Zeitmangel auf das Anzeigen bzw. Ausblenden von Informationen beim Anklicken der Stempelstelle. Die ursprüngliche Überlegung war, dass sobald eine Stempelstelle ausgewählt wurde, alle anderen Stempelstellen und Wanderrouen ausgeblendet werden, bis auf jene Routen, welche an der ausgewählten Stempelstelle vorbeiführen. Das Anzeigen der Stempelstelleninformationen, wie bspw. Name oder Höhe über Normalnull, war zwar auch angedacht, sollte aber nicht die alleinige Interaktionsmöglichkeit bleiben. Auf diesen Punkt, sowie weiterführende Interaktionsmöglichkeiten wird in Kapitel 4.3 Ausblick weiter eingegangen.

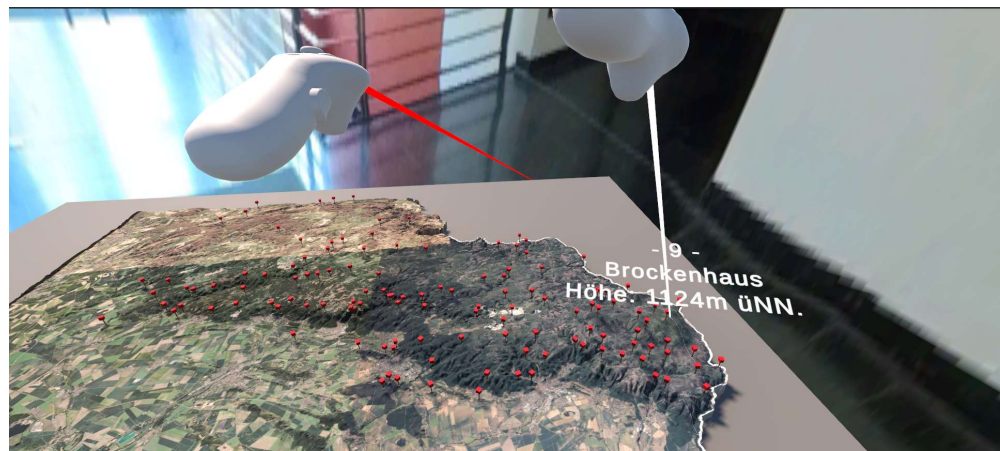


Abbildung 8: Interaktion mit Stempelstellen

2.5 Wanderrouten

Neben den Stempelstellen soll auch eine große Anzahl von Wanderrouten auf dem Modell vom Harz abgebildet werden.

2.5.1 Datengrundlage

Die Wanderrouten wurden von der Website touren.harzinfo.de entnommen, welche eine ungeschützte Schnittstelle verwendet, um Routen vom Backend des Mutterunternehmens Outdooractive AG zu beziehen. Über diese API konnte eine Liste von Harzer Wanderrouten akquiriert werden. Diese Wanderrouten (ca. 640) liegen als JSON-Dateien vor und mussten, ähnlich wie die Stempelstellen, vorverarbeitet werden, bevor sie in Unity eingelesen werden konnten. Das geschah mit dem Python-Skript „wanderroutenJsonsAnpassen.py“, welches die WGS84-Koordinaten in ETRS89-Koordinaten umwandelt. Nach der Vorverarbeitung wurden die JSONs in ein Konstrukt aus C#-Objekten eingelesen.

2.5.2 Darstellung und Platzierung

Die Darstellung der Wanderrouen erfolgt über Splines, welche die Liste der Wegpunkte der Wanderrouen als Knoten übergeben bekommen. Die Unity-Koordinaten der Knoten werden aus den Wegpunktkoordinaten mit Hilfe des CoordinateService errechnet, welcher die Skalierung und Überhöhung des Harz-Modells berücksichtigt.

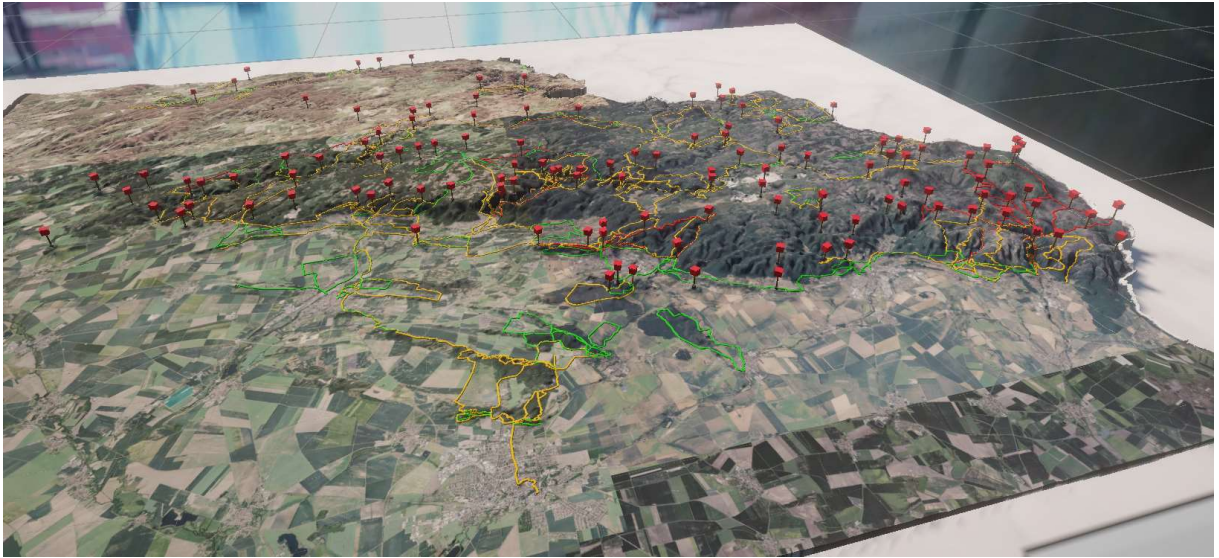


Abbildung 9: Finale Darstellung der Wanderrouen auf dem Modell in Unity

2.5.3 Interaktion

Ursprünglich sollte die Interaktion mit den Wanderrouen analog zur Interaktion mit den Stempelstellen erfolgen. Durch Anklicken einer Wanderroute sollten Informationen, wie Länge, Schwierigkeit, Höhenmeter und Bewertung der Route angezeigt werden. Zudem sollten alle anderen Wanderrouen, sowie alle Stempelstellen, welche nicht an der Route liegen ausgeblendet werden. Durch Zeitmangel / Fehlplanung konnte leider nur ersteres fertiggestellt werden, worauf im Kapitel 4 weiter eingegangen wird.

2.6 Umgebung

Damit die Umgebung des Modells, der Stempelstellen und der Wanderrouen nicht nur ein leerer Raum ist, wurde ein 360-Grad-Foto in die Szene platziert, welches das Modell umgibt. Das 360° Foto ist im Obergeschoss des Atriums von Haus 9 aufgenommen worden. Das Harz-Modell steht auf einem quadratischen Tisch.

3 Ergebnis

Die folgenden Kapitel geben eine Zusammenfassung des Projektzustands zum Zeitpunkt der Abgabe, beschreiben wie das Projekt zu installieren und bedienen ist und erklären Konfigurationsmöglichkeiten der Software.

3.1 Zielumsetzung

Wie in Kapitel 1.3 genannt, wurde das Projekt mit 5 verpflichteten, sowie 2 optionalen Zielen angetreten. In diesem Kapitel soll nun eine Bestandsaufnahme durchgeführt werden, um zu zeigen, welche dieser Ziele zum Zeitpunkt der Abgabe tatsächlich umgesetzt wurden. Folgende Tabelle listet alle Ziele und ihren Bearbeitungszustand auf.

Tabelle 1: Umsetzungsstatus der Projektziele (✓ = vollständig erfüllt, * = teilweise erfüllt, ✗ = nicht erfüllt)

Verpflichtende Ziele	#	Optionale Ziele	#
Erstellung eines dreidimensionalen Harz-Modells	✓	Einbinden von Google Street View	✗
Stempelstellen darstellen	✓	Routenfarbe je nach Schwierigkeitsgrad der Route ändern	✓
Wanderrouen darstellen	✓		
Interaktion mit Wanderrouen, um Informationen anzuzeigen	*		
Interaktion mit Stempelstellen, um Informationen anzuzeigen	*		

3.2 Inbetriebnahme

Um HIKE zu verwenden, muss das Projekt aus [2] heruntergeladen werden. Nach dem Extrahieren des Projekts kann dieses mithilfe des Unity Hubs geöffnet werden. Dafür muss ggf. noch die passende Unity-Version (2022.3.22f1) installiert werden. Aus dem Ordner Assets/Scenes im Project-Explorer muss dann die „LargeTerrainSzene“ ausgewählt und geladen werden. Daraufhin muss das Terrain und weitere Inhalte generiert werden. Als erstes muss der TerrainManager ausgewählt werden. Im Inspector sind daraufhin die Knöpfe „Regenerate“, „Clear“ und „Regenerate on Game Start“ zu finden. Clear löscht alle bestehenden Terrains, während Regenerate die Terrains neu generiert (und vorher Clear ausführt). Das Kästchen „Regenerate on Game Start“ sorgt dafür, dass (sofern das Häkchen gesetzt wurde) Regenerate beim Start des Spiels ausgeführt wird. Für alle drei Inhalts-

Manager muss in folgender Reihenfolge Regenerate ausgeführt werden: TerrainManager, StempelstellenManager, WanderrouenManager. Je nach Hardware kann dieser Prozess zwischen 15 Minuten bis zu einer Stunde dauern. Um die Ladezeit zu reduzieren, können Stempelstellen und Wanderrouen gefiltert werden. Im StempelstellenManager bzw. WanderrouenManager (Inspector) ist hierfür eine Liste vorhanden, in welche IDs eingetragen werden können. Die Checkbox „Filter Whitelist“ erlaubt hierbei das Umschalten zwischen Whitelist und Blacklist. Alternativ kann aber auch eine Datei mit IDs zugewiesen werden.

3.3 Bedienung

Die Bedienung ist denkbar einfach. Bewegung kann sowohl durch Translation im realen Raum, sowie durch Teleportation (indem der rechte Grip-Button gedrückt wird) erzielt werden. Stempelstellen können mit dem Controller selektiert werden, um Informationen anzuzeigen. Die Knöpfe auf dem Tisch können aktuell leider noch nicht genutzt werden.

3.4 Konfigurationsmöglichkeiten

HIKE ist so konzipiert, dass alle (vorverarbeitete) digitale Geländemodelle, Stempelstellen, sowie Wanderrouen dargestellt werden können. Hierfür gibt es einige Konfigurationsmöglichkeiten, welche in Tabelle 2 erläutert werden. Unter Edit > Project Settings > HikeSettingsProvider können diese Einstellungen in Unity angepasst werden.

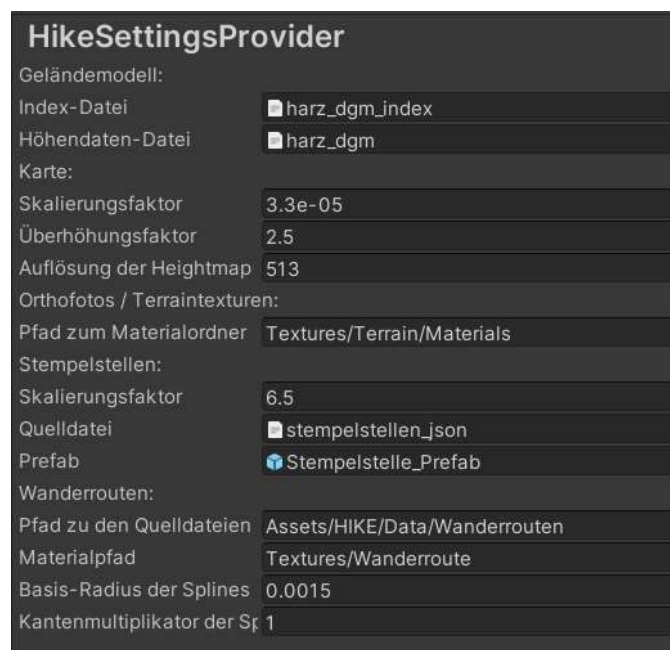


Abbildung 10: Ausschnitt aus dem Einstellungsfenster in Unity. Zu sehen ist der HikeSettingsProvider mit allen Konfigurationsmöglichkeiten.

Tabelle 2: Einstellungen in Unity im HikeSettingsProvider

Name	Funktion	Default
Geländemodell:		
Index-Datei	Eine .json Index-Datei mit Meta-Informationen über die Höhendaten-Datei.	harz_dgm_index.json
Höhendaten-Datei	Eine .bytes Datei mit Höhendaten (zwischen 0 und 1).	harz_dgm.bytes
Karte:		
Skalierungsfaktor	Bestimmt in welchem Größenverhältnis die Miniatur des Harzes zu den realen Dimensionen steht.	3.3e-05
Überhöhungsfaktor	Bestimmt einen Multiplikator, mit welchem die Höhe des Terrains skaliert.	2.5
Auflösung der Heightmap	Bestimmt die Anzahl der Samplepunkten pro Terrain. Dies beeinflusst die Anzahl an Terrainsegmenten, die zur Darstellung des gesamten Modells genutzt werden sollen.	513
Orthofotos / Terraintexturen:		
Pfad zum Materialordner	Lokaler Pfad unter Assets zum Ordner mit den DOP-Materialien.	Textures/Terrain/Materials
Stempelstellen:		
Skalierungsfaktor	Bestimmt um welchen Faktor die Miniatur der Stempelstelle skaliert wird.	6.5
Quelldatei	Eine .json Datei mit Stempelstelleninformationen.	stempelstellen_json.json
Prefab	Das Stempelstellen Prefab (GameObject).	Stempelstelle_Prefab
Wanderrouuten:		
Pfad zu den Quelldateien	Lokaler Pfad unter Assets zum Ordner mit den Wanderrouuten.	Assets/HIKE/Data/Wanderrouuten
Materialpfad	Lokaler Pfad unter Assets zum Ordner mit den Schwierigkeitsgrad-Materialien.	Textures/Wanderrouute
Basis-Radius der Splines	Bestimmt die Dicke der Splines.	0.0015
Kantenmultiplikator der Splines	Bestimmt den Detaillgrad der Splines, 1 entspricht 1 Knoten pro Samplepunkt, alles > 1 führt zu Interpolation, alles < 1 sorgt für Detaillverlust.	1

Dadurch, dass HIKE Inhalte dynamisch auf Basis von Daten generiert, besteht die Möglichkeit andere DGMs, Stempelstellen, Wanderrouten oder auch Texturen zu verwenden. Das DGM kann mit Hilfe des Skripts „dgmConverter.py“, welches im Ordner Assets/HIKE/Scripts/Python zu finden ist vorverarbeitet werden. Dazu muss lediglich das DGM im XYZ-Format und ETRS89-Koordinaten vorliegen, die Grenzen des DGMs im Skript (Dict-Variable „bounds“) definiert und der Pfad zu den DGM-Segmenten (Variable „directory“), sowie den Ausgabedateien angepasst werden. Die vom Skript generierten Dateien müssen dann in Unity importiert und in den Project-Settings zugewiesen werden (siehe Tabelle 2). Stempelstellen und Wanderrouten müssen als JSON-Dateien vorliegen und können somit auch angepasst werden. Die Koordinaten müssen jedoch immer ETRS89 sein. Um die Texturen des Terrains anzupassen kann das Python-Skript „textureMerger.py“ verwendet werden. Dabei muss lediglich der Pfad zum Ordner mit den rohen TIFF-Bildern und zum Ausgabeordner, sowie die tile_size definiert werden. Die Tile-Size s ergibt sich aus der Auflösung der Heightmap r minus 1 multipliziert mit der Gitterweite des DGMs w_{DGM} und der Gitterweite der Texturen w_{DOP} (Gitterweiten in Meter). $s = (r - 1) * w_{DGM} * w_{DOP}$.

Daraus ergibt sich beispielweise eine tile_size von 2560, für $r = 513$, $w_{DGM} = 5$, $w_{DOP} = 1$. Die vom Skript generierten Texturen müssen dann in den Ordner Assets\Resources\Textures\Terrain eingefügt werden. In Unity können dann, durch Auswählen dieser Texturen und Verwenden der Option Assets > Create Materials, Materialien für alle Texturen erzeugt werden. Diese Materialien werden automatisch nummeriert und beim Generieren des Terrains den Segmenten mit der gleichen Nummer zugewiesen.

4 Fazit

Wie in Kapitel 1.5 und 3.1 beschrieben, verlief die Projektumsetzung nicht wie ursprünglich geplant, weshalb nicht der volle Zielumfang erreicht werden konnte. Die folgenden Kapitel reflektieren diesen Umstand und seine Hintergründe und nennen Verbesserungsvorschläge für kommende Projekte.

4.1 Herausforderungen

Die schwerwiegendste Herausforderung bei der Umsetzung von HIKE war das zeitgleiche Anlernen und Verwenden unbekannter Technologien und Vorgehensweisen, wie bspw. Unity, Blender, Python, bzw. Programmierung in Unity oder Aufbau von Unity-Projekten. Dazu kam der Umstand, dass andere Lehrveranstaltungen bzw. Projekte ebenfalls umfangreiche zeitliche Zusagen erforderten, was die verfügbare Wochenarbeitszeit stark limitierte und durch den tatsächlichen Umfang des anfangs unterschätzten Projektumfangs potenziert wurde. Aber auch aus rein technischer Perspektive gab es Herausforderungen zu bewältigen. Die dynamische Darstellung der Höhendaten in Unity erwies sich beispielsweise als komplexer als gedacht, wie in 2.3.2 beschrieben wurde. Es gibt aber noch weitere Beispiele, wie dieser Projektschritt Probleme bereitet hat. Eine der versuchten Darstellungstechniken sah beispielweise die Verwendung von Heightmaps vor, welche als Bilddatei vorgeneriert und verwendet wird. Dies führte jedoch zu obskuren Ergebnissen, dessen Ursprung erst nach stundenlangem suchen (mehr als das eigentliche Wochenpensum), als Fehler in der Byteorder identifiziert werden konnte. Die Abbildung 11 zeigt den drastischen Unterschied, den diese eine Änderung erzeugt hat.

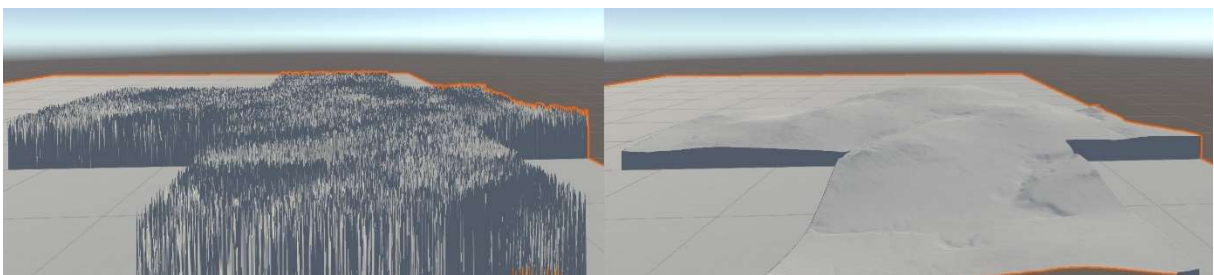


Abbildung 11: Unterschied zwischen Big Endian (links) und Little Endian (rechts)

Die letzte große Problematik, waren die Ladezeiten. Das Projekt nutzt mehrere Dutzend Gigabyte an Daten zur Erzeugung des Modells. Zudem werden für die Wanderrouten hunderte Meshs generiert. Jedes Neuladen der Szene, sei es nur wegen einer kleinen Code-Änderung,

zieht Ladezeiten von einigen Minuten nach sich. Dieser Umstand erschwert Tests und erhöht den sowieso schon hohen Zeitbedarf.

4.2 Eigenreflektion

Im Folgenden wird umfassend auf alle Aspekte, die zu dem vorliegenden Ergebnis geführt haben, eingegangen und persönliche Schlussfolgerungen gezogen.

4.2.1 Fabian

Nach anfänglich außerplanmäßig schnellem Fortschritt stagnierte die Fortschrittsrate zunehmend im Projektverlauf. Dieser Umstand führte dazu, dass einige Projektziele zwar erfüllt aber nicht zwingend zufriedenstellend umgesetzt wurden und das Projekt weitgehend unpoliert abgegeben werden musste. Diese Stagnation war basierte unter anderem auf dem Umstand, dass andere Projekte anderer Veranstaltungen im gleichen Zeitraum ähnlich große Zuwendung erforderten, was die wöchentlich verfügbare Zeit ausgedünnt hat. Dieser Umstand hätte im Zeitplan berücksichtigt werden müssen. Weiterhin kam hinzu, dass das Entwickeln und Testen der Software unvorhergesehen viel Zeit in Anspruch nahm, da jede Änderung, durch den großen Datenumfang, mit langen Ladezeiten verbunden war. Diesem Problem hätte durch die Verwendung einer leichtgewichtigen, bzw. Vereinfachten Testumgebung entgegengewirkt werden sollen. Diese Erkenntnis blieb allerdings bis kurz vor Ende des Entwicklungszeitraums aus. Grundsätzlich war die Projektidee gut gewählt, da sich das Projekt, wie von der Aufgabenstellung gefordert, umfassend mit der Darstellung externer Daten in Unity befasst hat. Allerdings kam durch den Zeitmangel die Interaktion mit diesen Daten zu kurz. Grundsätzlich wäre für die Umsetzung des Projekts mehr Zeit als geplant nötig gewesen. Einige Aufgaben dauerten deutlich länger als gedacht, während Zeit für Bugfixes, Polishing und Puffer nicht eingeplant waren. Rückblickend hätte das Projekt also deutlich besser geplant werden müssen. Dazu gehört auch der korrekte Umgang mit der geringen verfügbaren Zeit: Entweder hätte der Projektumfang kleiner gewählt werden sollen, oder es hätten mehr Projektziele optional statt verpflichtend sein sollen. Das Projekt hat, aus persönlicher Sicht gesprochen, jedoch ein verstärktes Interesse an VR-, Spiele- bzw. Simulatoren-Entwicklung, sowie Datenverarbeitung geweckt und war zudem eine hilfreiche Erfahrung in Punkto Projektplanung und -management.

4.2.2 Matthias

Das Projekt verlief zunächst schneller als geplant, die anfänglichen Aufgaben wurden also vor ihrem ursprünglich anberaumten Endzeitpunkt fertiggestellt. Zum Ende des Projektes wurde die Zeit je Aufgabe knapp, so dass das Projekt vermutlich nicht die Qualität und den Umfang hat, den mehr investierte Zeit zur Folge gehabt hätten. Der Unterschied beziehungsweise der Übergang zwischen dem guten Verlauf am Projektanfang und der Zeitknappheit am Projektende ist aus meiner Perspektive auf verschiedene Einflüsse zurückzuführen. Zunächst hätte ein geringerer Projektumfang dazu führen können, dass den verbleibenden Teilzielen mehr Arbeitszeit zugeordnet worden wäre. Daraus folgt, dass eine Verschiebung von verpflichtenden zu mehr optionalen Zielen hätte hilfreich sein können. Mehr Wochenarbeitszeit für das Projekt einzuplanen scheint unrealistisch, da diese Ressource auch durch andere Lehrveranstaltungen ausreichend in Anspruch genommen wurde. Ein weiterer Faktor, welcher zu außerplanmäßig erhöhtem Zeitaufwand und damit gegen Ende zu Zeitknappheit führte, war das Unterschätzen der Einarbeitungszeit und der Gesamtbearbeitungszeit der Aufgaben mit den Programmen Blender und Unity. Dieser Faktor ist zum einen auf persönliche Fehlplanung zurückzuführen, da vorher noch nicht mit diesen Programmen gearbeitet wurde. Zum anderen bestand die Bearbeitungszeit oft aus dem Rückgängigmachen der Arbeitsschritte, die aus irreführenden Quellen stammten. Die Lösung hierfür hätte im Einplanen von mehr Recherchezeit insgesamt bestehen können. Als letzter Faktor ist zu nennen, dass keine separate Zeit für das Einrichten der Programme eingeplant war. Bei Ladezeiten von über 30 Minuten, Problemen mit der Versionsverwaltung und keinerlei Erfahrung mit den einzurichtenden Programmen bei wechselnden Arbeitsplätzen wäre das vorteilhaft gewesen. Schlussendlich fällt mein Fazit des Projekts gemischt aus. Positiv zu bewerten sind die Arbeit des Projektpartners, der Erkenntnisgewinn bezüglich Ressourcenplanung und die eigene Arbeitsweise in der Anfangsphase des Projekts. Kritisch zu hinterfragen sind insbesondere die eigene Zeitplanung während der zweiten Projekthälfte, das Vertrauen auf (Internet-)Quellen bei der Nutzung von Blender und Unity sowie die eigene Kompetenz bei der realistischen Selbsteinschätzung.

4.3 Ausblick

HIKE bietet viel Spielraum zur Erweiterung, unter Anderem, da einige Projektziele, wie bspw. die Stempelstellen- und Wanderrouteninteraktion, noch nicht vollständig umgesetzt sind. Eine mögliche Fortführung des Projekts könnte also folgende Schritte beinhalten: Erstens, Umstellung der Architektur auf ein eventbasiertes System (Observer Entwurfsmuster). Zweitens, Erfüllen der Projektziele Stempelstellen- und Wanderrouteninteraktion in ursprünglich geplantem Umfang. Drittens, Verbesserung der VR-Steuerung, wie bspw. Bewegung. Viertens, Verschönerung der Umgebung. Hier könnte beispielsweise ein besseres 360° Bild verwendet, oder ein Raum dreidimensional nachmodelliert werden. Fünftens, Verbesserung der Kartendetails, beispielsweise durch automatische Erzeugung von Häuser-Modellen oder Platzierung von Bäumen. Hier würde sich auch die Verwendung des digitalen Oberflächenmodells (DOM) als Normalmap der Terraintexturen anbieten. Parallel sollte die Optimierung des Projekts, sowie die Implementierung einer leichtgewichtigen Testinfrastruktur im Vordergrund stehen. Des Weiteren kann die Anwendung um Funktionen, wie bspw. einen Routenplaner, oder die geplante Google Street View Integration erweitert werden. Schlussendlich wäre auch eine Erweiterung um andere Wandergebiete, wie bspw. Teile der Alpen, oder dem Elbsandsteingebirge denkbar. HIKE hat ausreichend Potenzial für Erweiterungen, ist in der Praktikabilität für den Normalverbraucher jedoch beschränkt, weshalb die Nützlichkeit dieser Erweiterungen diskutiert werden müsste.

III. Quellen- / Ressourcenverzeichnis

- [1] M. Nagel, „Harzer Wandernadel,“ [Online]. Available: <https://www.harz-wandernadel.de/stempelstellen.php>. [Zugriff am 04 April 2024].
- [2] F. Siemens, „lbfs.eu,“ 18 Juli 2024. [Online]. Available: http://www.lbfs.eu/download/HIKE_mit_Ressourcen_kompakt.zip. [Zugriff am 18 Juli 2024].
- [3] LVerGeo, „Kostenfreies Digitales Geländemodell mit einer Rasterweite von 5m (DGM5),“ Landesamt für Vermessung und Geoinformation, [Online]. Available: <https://www.lvermgeo.sachsen-anhalt.de/de/gdp-dgm5.html>. [Zugriff am 04 April 2024].
- [4] Harzinfo, „Touren - Plane dein nächstes Abenteuer,“ Outdooractive AG, [Online]. Available: <https://touren.harzinfo.de/de/tours>. [Zugriff am 11 April 2024].

IV. Anhang

Originaler Zeitplan: „Zeitplan_Original_Anwendungspraktikum_XR.xmls“

Finaler Zeitplan: „Zeitplan_Final_Anwendungspraktikum_XR.xmls“

Stempelstellen C++ Skript: „StempelstellenHWN_Cpp.zip“